



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Патрик Барши

Штатичка анализа Go кода у циљу читљивости и одржавања

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Патрик Барши	Број индекса:	SV7/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Штатичка анализа Go кода у циљу читљивости и одржавања
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	Монографска документација
Тип записа, ТЗ :	Текстуални штампани материјал
Врста рада, ВР :	Дипломски - бечелор рад
Аутор, АУ :	Патрик Барши
Ментор, МН :	Др Игор Дејановић, редовни професор
Наслов рада, НР :	Штатичка анализа Go кода у циљу читљивости и одржавања
Језик публикације, ЈП :	српски/ћирилица
Језик извода, ЈИ :	српски/енглески
Земља публикавања, ЗП :	Република Србија
Уже географско подручје, УГП :	Војводина
Година, ГО :	2026
Издавач, ИЗ :	Ауторски репринт
Место и адреса, МА :	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	4/29/11/2/2/0/2
Научна област, НО :	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :	статичка анализа, линтер, Go, Rust, Tree-sitter, конкретно стабло синтаксе, дијагностике
УДК	
Чува се, ЧУ :	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	У овом раду представљени су дизајн и имплементација алата за статичку анализу изворног кода написаног у програмском језику Go. Алат је имплементиран у програмском језику Rust и користи Tree-sitter као позадински систем за парсирање, чиме се добија конкретно стабло синтаксе анализираног кода. За разлику од анализатора интегрисаних у компајлер, овај приступ одржава алат независним од Go инфраструктуре. Имплементирано је 21 правило статичке анализе подељено у две категорије: општа правила која детектују честе програмерске грешке и правила изведена из књиге <i>100 Go Mistakes and How to Avoid Them</i> аутора Теиве Харсањија. Свако правило независно обрађује стабло синтаксе и производи дијагностике са прецизним информацијама о позицији у изворном коду. Алат је доступан као самостална апликација командне линије способна да анализира појединачне фајлове или целе директоријуме.
Датум прихватања теме, ДП :	01.01.2025
Датум одбране, ДО :	01.01.2025
Чланови комисије, КО :	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
	Члан, ментор: Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Patrik Barši
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Static Go code analysis for readability and maintainability
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	4/29/11/2/2/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	static analysis, linter, Go, Rust, Tree-sitter, concrete syntax tree, diagnostics
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents the design and implementation of a static analysis tool for source code written in the Go programming language. The tool is implemented in Rust and uses Tree-sitter as its parsing backend to produce a Concrete Syntax Tree of the analyzed code. Unlike compiler-integrated analyzers, this approach keeps the tool decoupled from the Go toolchain. The implementation covers 21 linting rules divided into two categories: general rules targeting common programming mistakes, and rules derived from the book <i>100 Go Mistakes and How to Avoid Them</i> by Teiva Harsanyi. Each rule operates independently on the syntax tree and emits diagnostics with precise source locations. The tool is provided as a standalone command-line application capable of analyzing individual Go source files or entire directory trees.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	
President:	Petar Petrović, Phd., assoc. professor
Member:	Marko Marković, Phd., asist. professor
Member:	
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Шта је Turpst?	1
1.2	Припрема текста	3
1.3	Дефинисање појмова и преводи	3
1.4	Стил писања	3
1.5	Граматика	3
1.6	Слике и табеле	4
1.7	Листа за дораде	4
1.8	Цитирање	5
1.9	Предаја рада	6
1.10	Очекивани обим рада	6
2	Стање у области	7
2.1	go vet	7
2.2	staticcheck	7
2.3	revive	7
2.4	golangci-lint	7
2.5	Поређење постојећих алата	7
2.6	Tree-sitter	8
2.7	Rust у статичкој анализи	8
3	Архитектура	9
3.1	Компоненте система	9
3.2	Rule интерфејс	9
4	Закључак	11
	Списак слика	13
	Списак листинга	15
	Списак табела	17
	Списак коришћених скраћеница	19
	Списак коришћених појмова	21
	Биографија	23
	Литература	25
	Грешке у литературним наводима	27
	TODOs	29

Овај шаблон је намењен за припрему завршних дипломских и мастер радова на студијском програму за софтверско инжењерство и информационе технологије на Факултету техничких наука. Урађен је употребом **Typst** система који је описан у наставку.

1.1 Шта је Typst?

Typst је систем за припрему, односно аутоматски прелом докумената (енг. *typesetting*), и посебно је погодан за техничке и научне текстове. Омогућава прецизно форматирање, математичке формуле, библиографије и сложене структуре. Користи `.typ` текстуалне фајлове и компајлира их у PDF.

Овакав приступ израде докумената називамо “What You See Is What You Meant” (оно што видиш је оно што си мислио) који је популаран у техничким доменима. На пример, *markdown* који користимо на GitHub-у и другим сајтовим је добар пример овог приступа. Насупрот овом приступу имамо “What You See Is What You Get” (оно што видиш је оно што и добијеш).

Разлике између ова два приступа су следеће:

- **WYSIWYG** (*What You See Is What You Get*) – Офис алати (нпр. Word, Google Docs):
 - ▶ Видиш тачно како ће документ изгледати приликом уређивања,
 - ▶ Фокус на визуелном изгледу (фонт, боје, размаци),
 - ▶ Мање аутоматизације за сложене структуре (референце, садржај).
- **WYSIWYM** (*What You See Is What You Meant*) – Typst:
 - ▶ Уноси се *логичка структура* (наслови, секције, математичке формуле), а изглед се одређује касније (преко стилова). Пример:
= Увод
Ово је `_логичка_` ознака наслова, а не визуелни избор фонта.
 - ▶ Детаљнија контрола (нпр. табеле, формуле),
 - ▶ Аутоматско генерисање садржаја, библиографије, референци у тексту,
 - ▶ Аутоматско повезивање референци са делом текста који се референцира,
 - ▶ Аутоматска нумерација поглавља, секција, слика,
 - ▶ Аутоматско генерисање индекса слика, табела, листинга,
 - ▶ Аутоматско распоређивање слика, табела, прелом страна, прелом речи на крају реда итд.
 - ▶ Могућност скриптовања – нпр. аутоматско пребројавање поглавља, слика, табела и сл. у кључној документацијској информацији код завршних радова,
 - ▶ Могућност верзионисања — `.typ` фајлови су обични текст, па се лако прате промене (нпр. са Git).
 - ▶ Стабилност — боље рукује великим документима (нпр. књиге, дисертације, завршни радови).

Typst је слободан софтвер отвореног кода.

1.1.1 Употреба Typst алата

После инсталације¹ довољно је да из фолдера овог шаблона позовете следећу команду:

```
typst watch zavrzni-rad.typ
```

Ова команда покреће Typst у моду у коме прати промене над фајловима и при сваком промени регенерише PDF фајл.

Отворите PDF фајл `zavrzni-rad.pdf` у неком од прегледача који освежава приказ на сваку промену без губљења позиције (нпр. `Okular`² или `Sumatra PDF`³). Сада у вашем текстуалном едитору мењајте `.typ` фајлове. Видећете да се PDF приказ готово тренутно ажурира.

1.1.2 Фонтови

Овај шаблон користи Liberation фонт па се побрините да вам је исправно инсталиран. Да бисте проверили који фонтови су доступни позовите команду:

```
typst fonts
```

или, да проверите да ли имате Liberation фонт инсталиран:

```
typst fonts | grep -i liberation
```

1.1.3 Упутство за фајлове у Typst шаблону

У фајлу `metadata.typ` налазе се метаподаци које је потребно да попуните и који ће се аутоматски применити у свим деловима текста где је то потребно. На пример, ту дефинишете:

- формат стране (a4 или iso-b5),
- наслов рада,
- име аутора,
- ментора,
- апстракт,
- кључне речи,
- чланове комисије за оцену и одбрану
- итд.

Такође, дефинишете и потребне елементе на енглеском језику.

Шаблон је прављен тако да није потребно да се ручно мењају задатак, кључна документацијска информација, насловна страна. Све измене се спроводе кроз варијабле у фајлу `metadata.typ`.

У фајлу `biografija.typ` пишете своју кратку биографију.

У фајлу `literatura.bib` уносите референце у BibTeX формату.

У фолдеру `rogavlja` пишете текст вашег рада. Свако ново поглавље морате да укључите са `#include` директивом у дну `zavrzni-rad.typ` фајла.

У фолдеру `slike` смештате слике у PNG формату.

Остале фајлове не треба да дирате.

Шаблон подржава и A4 и ISO-B5 формат. Прилагођен је двостраној штампи. Пошто се прелом обавља аутоматски можете мењати по потреби и није потребно унапред да одлучите да ли ћете користити A4 или ISO-B5.

¹<https://github.com/typst/typst#installation>

²<https://okular.kde.org/>

³<https://www.sumatrapdfreader.org/free-pdf-reader>

1.2 Припрема текста

- За припрему рада се користи `Typrst + BibTeX` за референце по узору на овај шаблон.
- Направите клон (енг. *fork*) овог шаблона и додајте ментора као колаборатора.
- Текст се пише **искључиво на ћирилици**. За пресловљавање у `Typrst` можете користити Ћирко [1] који се може интегрисати са разни текстуалним едиторима.
- Текст задатка и чланове комисије ћете добити од ментора.
- Обавезно инсталирајте подршку за српски језик са провером правописа у свом текстуалном едитору.
- Обавезно прочитајте рад у целини бар једном пре слања ментору на читање.
- За мастер рад потребно је написати и рад за Зборник радова ФТН-а. Обим је 4 странице двостубачно. Погледајте пример. Шаблон преузимате са сајта Зборника⁴.

1.3 Дефинисање појмова и преводи

- При првом увођењу одређеног појма, описати га и евентуално навести назив на енглеском.

Пример:

Насупрот језицима опште намене, језици специфични за домен (ЈСД, енг. *Domain-Specific Languages*) представљају рачунарске језике који нуде повећање експресивности кроз употребу концепата и нотација прилагођених и често ограничених на одређени домен проблема.

- Користити уобичајени превод уколико постоји или назив у оригиналу у курзиву (енг. *italics*).
- Акроними настали од енглеских речи се пишу на латиници (нпр. HTML, DSL, CSS, AI).
- *Class diagram* се преводи као "дијаграм класа" а не као "класни дијаграм".

1.4 Стил писања

- Знаци интерпункције:
 - Тачке, зарези, двотачке – пишу се уз претходну реч. Следећа реч почиње после размака.
 - Цртица: пише се са размаком са обе стране.
 - Заграде: пише се уз прву односно последњу реч у загради. Отворена заграда почиње после размака у односу на претходну реч. После затворене заграде такође иде размак.
- Код техничких текстова се обично избегава прво лице једине и користе пасивни облици где год је то могуће. Нпр. уместо “Ја сам имплементирао” је боље рећи “У раду је приказана имплементација...”.
- Избежавати квалификације:
 - “**Веома** је тешко направити...”
 - “Информациони систем је **страховито** сложен...”
 - “Имплементација је **јак**о компликована...”

1.5 Граматика

- “Бисмо, бисте” – пише се спојено.
- “Не” – пише се одвојено ако стоји уз глагол, али спојено са остатком речи ако је у оквиру придева или прилога.

Пример:

- Придеви и прилози: “недовољно, невидљиво, неразговорно, неизграђен...”.

⁴<http://www.ftn.uns.ac.rs/ojs/index.php/zbornik>

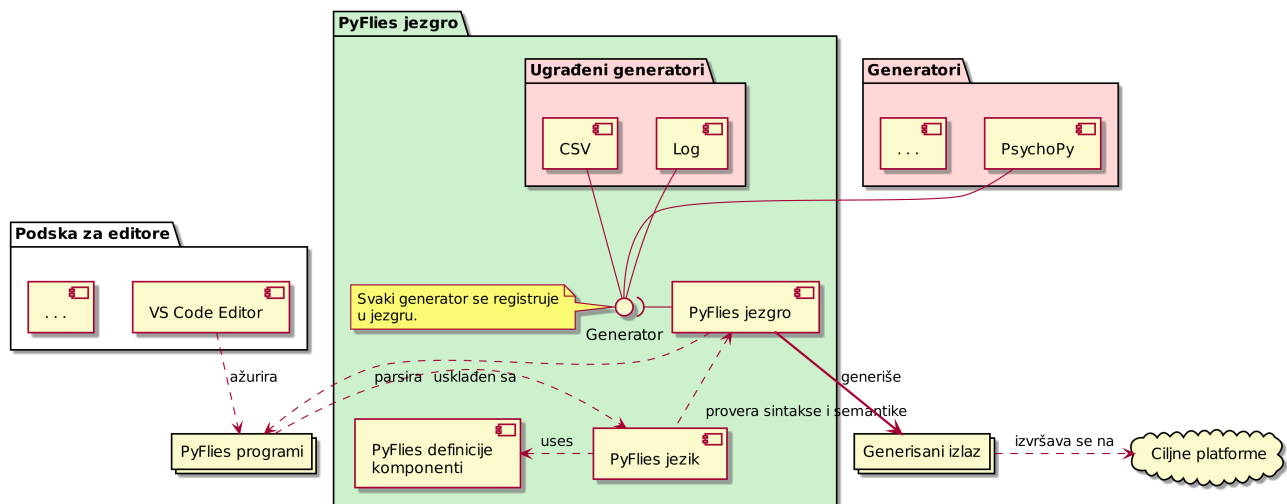
- Глаголи: “не видим, не могу...”.
- На почетку набрајања ставити двотачку.
- Не користити енглеску конвенцију за писање наслова. У српском језику само прва реч у наслову почиње великим словом.

1.6 Сlike и табеле

- За сlike које представљају програмски код из едитора избежавати тамне теме. Светле теме боље изгледају у штампи и троше мање тонера. Најбоље је програмски код наводити као листинг у Turst-у јер ће онда бити конзистентно приказан у смислу фонтова и бојења синтаксе.
- Сlike и табеле морају имати наслов.
- Свака сlike и табела се морају барем једном цитирати у тексту.

Пример:

На слици 1 приказана је PyFlies архитектура која прати стандардне компајлерске архитектуре.



Слика 1: PyFlies архитектура.

Табеле се наводе на следећи начин:

Табела 1: Пример табеле

Податак 1	Податак 2
Вредност 1	Вредност 2

А цитирају у тексту овако 1. Можете приметити да ће се списак табела аутоматски креирати на крају на страници [Списак табела](#).

1.7 Листа за дораде

Овај шаблон има подршку за вођење TODO листе која се може користити од стране кандидата док пише текст али и од стране ментора приликом рецензије. Било где у тексту се може навести функција

`#todo`[Нешто што треба да се уради]

Као на пример овде **TODO: Објаснити како се користи todo функција**. И на крају рада ће бити генерисан [списак TODO уноса](#).

1.8 Цитирање

- Цитирајте све коришћене изворе на месту употребе.
- Не преузимајте садржаје дословно (*copy-paste*) већ својим речима препричајте прочитано и пробајте да укрстите са више литературних извора.
- Цитате наводите и у наслову слика које сте преузели. При преузимању слике потребно је да лиценца омогућава употребу у другим радовима.
- Све што не цитирате, а преузели сте из других извора, сматра се плагијатом.
- Цитирање се обавља навођењем броја литературног навода у угластим заградама.
- Цитат је део реченице и стога се наводи пре тачке која завршава реченицу.
- Цитат се у реченици понаша као реч, односно одваја се од околних речи.
- Можете истовремено цитирати више радова. На пример: [7,14] или [1-3,7,12].

Примери:

Данас су најпознатије проширена Бакус-Наурова форма (енг. *Extended Backus–Naur form - EBNF*) [13] и аугментована Бакус-Наурова форма.

...

...дефиниција дата је у RFC 5234 документу [14].

- Сви литературни наводи морају имати аутора, наслов, издавача и годину.
- Сви онлајн извори обавезно морају да имају наслов, линк до извора и датум приступа. За онлајн изворе није потребно наводити аутора, издавача и годину уколико нису познати.

Примери:

...

[69] A. Aho, J. Ullman, The theory of parsing, translation, and compiling, Vol. 1 of Series in Automatic Computation, Prentice-Hall, 1972.

...

[72] EBNF: A notation to describe syntax, Online, <https://www.ics.uci.edu/~pattis/misc/ebnf2.pdf>, 2013, приступ: 2021-08-01.

- Радове и књиге можете потражити на Гугл претраживачу за радове⁵. Испод сваке референце имате број цитата (већи број обично значи релевантнији рад), радове који га цитирају (помаже за даљу претрагу) и начин цитирања. Такође, можете директно преузети референцу у BibTeX формату и убацити је у фајл `literatura.bib`.
- Ако можете да бирате увек преферирајте дела који су прошла процес рецензије (радове, књиге), у односу на нерецензиране изворе (веб сајтови, блогови, радне верзије књига и радова, слајдови итд.).

1.8.1 Цитирање употребом BibTeX

BibTeX је текстуални језик/формат за дефинисање референци. Омогућава конзистентно и аутоматско нумерисање и форматирање референци.

⁵<https://scholar.google.com/>

Литературне наводе чувате у фајлу `literatura.bib` и у тексту их референцирате са `@ID` где `ID` представља једнозначни идентификатор референце. На пример, ако у `bib` фајлу имате литературни навод:

```
@book{dejanovic2021c,  
  author = {Игор Дејановић},  
  title = {Језици специфични за домен},  
  publisher = {Факултет техничких наука, Нови Сад},  
  year = {2021}  
}
```

Листинг 1: Пример BibTeX референце

У тексту ћете ово дело цитирати са `@dejanovic2021c` чиме ћете добити овакву референцу [2]. Можете приметити да се у рендерованом PDF-у приказује хиперлинкована референца на литературу на крају рада која је форматирана на прописан начин. Секција са литературом се аутоматски генерише на основу онога што је цитирано у тексту рада. Нумерација се такође аутоматски обавља тако да додавање нових референци, премештање текста итд. неће пореметити исправно бројање и повезивање.

Идентификатор може бити произвољан али је препоручено да буде облика “<презиме аутора><година>”. Ако за аутора имате више радова из исте године додајете суфикс као у претходном примеру.

Претходни блок BibTeX кода на листингу 1 је такође пример како можете у раду користити изворни код. Погледајте изворни код овог шаблона да видите како је наведен код и како се цитира.

Ово је пример цитирања научног рада [3], а ово је пример цитирања онлајн извора [4].

Уколико у BibTeX референцама недостаје неко поље то ће бити пријављено у секцији [Грешке у референцама](#).

Такође, можете се референцирати и на секције и поглавља у оквиру рада. На пример, овако се цитира поглавље 4 (Закључак).

1.9 Предаја рада

- За библиотеку је потребан један тврдо коричени примерак.
- Примерци за комисију — у договору са ментором.
- После одбране, ментор потписује рад за библиотеку. Потребан је и потпис руководиоца студијског програма. Потписан рад кандидат односи у библиотеку ФТН-а као и PDF верзију рада на USB диску.

1.10 Очекивани обим рада

- Дипломски: 40+ страна B5 или 30+ страна A4. Фонт 11.
- Мастер: 60+ B5 или 50+ A4. Фонт 11.

Статичка анализа кода (енг. *static analysis*) представља технику испитивања изворног кода без његовог извршавања. За разлику од динамичке анализе, која прати понашање програма током извршавања, статичка анализа открива потенцијалне грешке, кршења стилских конвенција и лоше праксе непосредно из изворног кода програма. Алата који обављају ову врсту анализе називају се линтери (енг. *linters*).

У екосистему програмског језика Go постоји неколико устаљених алата за статичку анализу. У наставку су описани најзначајнији, уз осврт на начин на који приступају анализи кода.

2.1 go vet

`go vet` је алат за статичку анализу уграђен директно у стандардну инсталацију Go језика. Анализира изворни код употребом апстрактног стабла синтаксе (АСС, енг. *Abstract Syntax Tree*) и информација о типовима добијених од Go компајлера. Фокусиран је на детектовање конструкција које су синтаксно исправне, али врло вероватно погрешне — на пример, погрешан формат аргумената у позивима функција попут `fmt.Printf`, или поређење структура које не смеју бити поређене.

Будући да је `go vet` тесно везан за Go компајлер, не може се користити независно од Go инфраструктуре [5].

2.2 staticcheck

`staticcheck` је тренутно најзаступљенији самостални линтер за Go. Покрива широк спектар провера — од откривања грешака и неискоришћеног кода до дијагностика специфичних за поједине пакете из стандардне библиотеке. Такође користи АСС и информације о типовима из Go компајлера, чиме постиже висок степен прецизности [6].

2.3 revive

`revive` је конфигурабилан алат за статичку анализу Go кода са нагласком на проверу стила и добрих пракси. Нуди могућност укључивања и искључивања појединачних правила, као и подршку за прилагођена правила путем додатака (енг. *plugins*). Такође зависи од Go компајлерске инфраструктуре [7].

2.4 golangci-lint

`golangci-lint` није самосталан линтер, већ збирка (енг. *aggregator*) која обједињује велики број постојећих Go линтера, укључујући `staticcheck` и `revive`, и омогућава њихово паралелно извршавање уз централизовану конфигурацију. Широко је коришћен у CI/CD окружењима [8].

2.5 Поређење постојећих алата

Сви наведени алата ослањају се на АСС и информације о типовима добијене из Go компајлера. Ово им омогућава висок степен прецизности, али их чини нераздвојиво везаним за Go инфраструктуру — нису употребљиви изван Go екосистема и не могу се лако прилагодити другим језицима.

Табела 2: Поређење алата за статичку анализу Go кода

Алат	Тип стабла	Независан од Go	Проширив
go vet	ACC	Не	Ограничено
staticcheck	ACC	Не	Не
revive	ACC	Не	Да
golangci-lint	—	Не	Да
golint (овај рад)	KCC	Да	Да

Из табеле 2 може се уочити да се алат развијен у оквиру овог рада разликује по употреби конкретног стабла синтаксе (KCC) и независности од Go компајлерске инфраструктуре, о чему је детаљније реч у поглављу 3.

2.6 Tree-sitter

Tree-sitter је библиотека за инкрементално парсирање изворног кода, написана у програмском језику C. Омогућава генерисање парсера за произвољне програмске језике на основу граматике дефинисане у JavaScript-у, при чему се добијени парсер компајлира у ефикасан C код. Резултат парсирања је конкретно стабло синтаксе — структура која чува сваки токен из оригиналног кода, укључујући размаке, коментаре и интерпункцију.

Кључна предност Tree-sitter-а је независност од конкретног програмског језика — иста библиотека и исти API употребљавају се за Go, Rust, Python, JavaScript и друге језике. Tree-sitter се такође широко користи у текстуалним едиторима (Neovim, Helix, Zed) за осветљавање синтаксе (енг. *syntax highlighting*) и навигацију кодом [9].

2.7 Rust у статичкој анализи

Rust је системски програмски језик фокусиран на сигурност меморије и конкурентност без употребе сакупљача смећа (енг. *garbage collector*) [10]. У последње време све је заступљенији у изградњи алата за развој софтвера (енг. *developer tooling*) — примери укључују ripgrep, fd, еха и компајлер rustc сам по себи.

За потребе овог рада, Rust је одабран јер нуди:

- квалитетне Rust везиве (енг. *bindings*) за Tree-sitter библиотеку,
- изражајан систем типова погодан за моделовање правила анализе,
- ефикасно извршавање без управљачке меморије.

Алат је организован као вишефазни пајплајн за статичку анализу. Свака фаза обавља јасно дефинисан задатак и предаје свој излаз следећој фази, чиме је постигнута модуларност и могућност независног тестирања сваке компоненте. **TODO: Додати слику архитектуре**

Go изворни код → Парсирање (Tree-sitter) → КСС → Примена правила → Дијагностике

3.1 Компоненте система

Систем се састоји од следећих компоненти:

- **parser** — учитава .go фајл са диска и позива Tree-sitter да га парсира у конкретно стабло синтаксе (КСС),
- **rules** — скуп независних модула, сваки имплементира интерфејс Rule,
- **linter** — агрегира сва правила и покреће их над стаблом,
- **diagnostics** — структура која чува информације о откривеним проблемима,
- **cli** — обрађује аргументе командне линије.

3.2 Rule интерфејс

Свако правило имплементира следећи Rust трејт (енг. *trait*):

```
pub trait Rule {  
    fn name(&self) -> &'static str;  
    fn check(&self, tree: &Tree, source: &str, file: &str) -> Vec<Diagnostic>;  
}
```

Захваљујући овом дизајну, додавање новог правила своди се на креирање новог модула и регистровање у листи унутар структуре Linter. Постојећа правила остају непромењена.

TODO: Додати архитектурни дијаграм са компонентама и токовима података

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	PyFlies архитектура.	4
---------	---------------------------	---

Списак листинга

Листинг 1 Пример BibTeX референце	6
---	---

Списак табела

Табела 1	Пример табеле	4
Табела 2	Поређење алата за статичку анализу Go кода	8

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] И. Дејановић, “Ћирко - ћирилично-латинични конвертор.” Accessed: August 8, 2025. [Online]. Доступно на <https://github.com/igordejanovic/cirko>
- [2] И. Дејановић, *Језици специфични за домен*. Факултет техничких наука, Нови Сад, 2021.
- [3] I. Dejanović, R. Vadera, G. Milosavljević, and Ž. Vuković, “TextX: A Python tool for Domain-Specific Languages implementation,” *Knowledge-Based Systems*, vol. 115, pp. 1–4, 2017, doi: [10.1016/j.knosys.2016.10.023](https://doi.org/10.1016/j.knosys.2016.10.023).
- [4] I. Dejanović, “A Domain-Specific Language (DSL) for designing experiments in psychology.” [Online]. Доступно на <https://github.com/pyflies/pyflies/>
- [5] “go vet — Go Command Documentation.” Accessed: June 20, 2026. [Online]. Доступно на <https://pkg.go.dev/cmd/vet>
- [6] “Staticcheck — The Advanced Go Linter.” Accessed: June 20, 2026. [Online]. Доступно на <https://staticcheck.dev/>
- [7] “revive — Fast, Configurable, Extensible Linter for Go.” Accessed: June 20, 2026. [Online]. Доступно на <https://revive.run/>
- [8] “golangci-lint — Fast Go Linters Runner.” Accessed: June 20, 2026. [Online]. Доступно на <https://golangci-lint.run/>
- [9] “Tree-sitter — An Incremental Parsing System for Programming Tools.” Accessed: June 20, 2026. [Online]. Доступно на <https://tree-sitter.github.io/tree-sitter/>
- [10] “Rust Programming Language.” Accessed: June 20, 2026. [Online]. Доступно на <https://rust-lang.org/>

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `pyflies` недостаје поље `urldate`

TODOs

1. Објаснити како се користи `todo` функција.
2. Додати слику архитектуре
3. Додати архитектурни дијаграм са компонентама и токовима података